Lecture 23

Pete Manolios
Northeastern

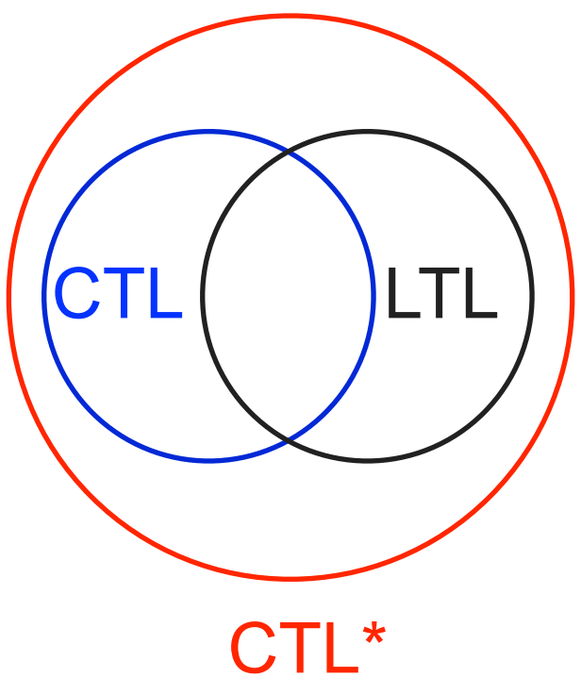
Temporal Logic: CTL*

- The syntax of CTL* formulas:
  - *e*, where *e* is an *expression*
  - $f \wedge g$ and $\neg f$, where *f*, *g* are formulas
  - **X** *f*, *f* **U** *g*, **E** *f*, where *f*, *g* are formulas
- The semantics of CTL* formulas wrt M, σ
  - $M, \sigma \models e$ iff $L(\sigma_0)$ $[\![e]\!]$
  - $M, \sigma \models f \wedge g$ iff $M, \sigma \models f$ and $M, \sigma \models g$
  - $M, \sigma \models \neg f$ iff it is not the case that $M, \sigma \models f$
  - $M, \sigma \models \mathbf{X}\, f$ iff $M, \sigma^1 \models f$
  - $M, \sigma \models f\, \mathbf{U}\, g$ iff $\exists\, i$ s.t. $M, \sigma^i \models g$ and $\forall\, j < i$, $M, \sigma^j \models f$
  - $M, \sigma \models \mathbf{E}\, f$ iff $\exists\, \delta = \langle \sigma_0, \ldots \rangle$ in M s.t. $M, \delta \models f$
- $M \models f$ iff $\forall$ fullpaths σ starting from an initial state: $M, \sigma \models f$

Temporal Logic Hierarchy

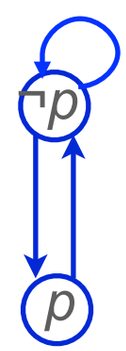
- CTL* subsumes CTL and LTL
- Find an LTL formula not expressible in CTL **A**(**FG** p) **??**
- Find a CTL formula not expressible in LTL **AG**(**EF** p) **??**
- Find a CTL* formula not expressible in LTL or CTL

A(**FG** p) ∨ **AG**(**EF** p)



A(**FG** p) holds

AF(**AG** p) does not



AG(**EF** p) holds

LTL only considers fullpaths, so *¬p, ¬p, ¬p, ...* can provide no evidence for **EF**p

Lattice Theory

Recall that a lattice $\langle L, \leq \rangle$ consists of a set L and a partial order $\leq$ on L such that any pair of elements has a meet ($\wedge$) and join ($\vee$). From a more algebraic, but equivalent, viewpoint, a lattice is a triple $\langle L, \wedge, \vee \rangle$ where $\wedge$ and $\vee$ satisfy the associative, commutative, idempotency, and absorption laws.

$$
\begin{array}{rrcl}
\text{(associative law)} & (a \vee b) \vee c & = & a \vee (b \vee c) \\
\text{(commutative law)} & a \vee b & = & b \vee a \\
\text{(idempotency law)} & a \vee a & = & a \\
\text{(absorption law)} & a \vee (a \wedge b) & = & a
\end{array}
$$

Each law also has a dual, which is obtained by interchanging $\vee$ and $\wedge$. We can then define $a \leq b \;\equiv\; (a \wedge b) = a$ and it follows that $a \leq b \;\equiv\; (a \vee b) = b$. We will stick to the algebraic view. Note that in light of associativity and commutativity, we do not need parentheses for sequences of joins or meets.

A lattice $\langle L, \vee, \wedge \rangle$ is *complete* iff $\vee S$, $\wedge S$ is defined for all $S \subseteq L$.
$f : L \to L$ is *monotonic* (*order preserving*) iff $\langle \forall x, y \in L :: x \leq y \to f.x \leq f.y \rangle$.
x is a *fixpoint* of f iff $f.x = x$.
f^n is defined as: f^0 is the identity function and $f^{n+1} = f \circ f^n$.

Note: f is monotonic implies that f^n is monotonic: The identity function is monotonic and composing two monotonic functions gives a monotonic function.

Tarski-Knaster

Theorem 1 *(Tarski-Knaster) Let f be a monotonic function on $\langle L, \vee, \wedge, \leq \rangle$, a complete lattice. Let $S = \{b \mid f.b \leq b\}, \alpha = \wedge S$. Then α is the least fixpoint of f.*

Proof (1: $f.\alpha \leq \alpha$) $\forall x \in S, \alpha \leq x$, thus $f.\alpha \leq f.x \leq x$. So, $f.\alpha \leq \wedge S$, *i.e.*, $f.\alpha \leq \alpha$.

(2: $\alpha \leq f.\alpha$) By monotonicity applied to (1), we get $f(f.\alpha) \leq f.\alpha$, so $f.\alpha \in S$, so $\alpha \leq f.\alpha$.

Obviously α is a fixpoint and below all fixpoints (they are all in S), thus the least fixpoint. $\square$

One can similarly define the greatest fixpoint, and it turns out that the least fixpoint can be obtained by starting with $\perp$ and applying f until a fixpoint is reached. This will require at most κ steps, where $\kappa = |L|^+$, the next cardinal after $|L|$, because each application of f that does yield a fixpoint will generate an element greater than all previous elements and that can't happen κ times, as there is no surjection from $|L|$ to κ $(= |L|^+)$. Recall that cardinals are just ordinals, but cardinal arithmetic is defined differently than ordinal arithmetic, *e.g.*, w^+ is the ordinal Ω, not $\omega + 1$. For finite lattices, we are guaranteed to reach a fixpoint if we iterate over w, the natural numbers, but for countably infinite systems, we may need to iterate way past w. As an exercise, construct such a lattice with cardinality w and a monotonic function f which requires more that w steps to reach a fixpoint.

Tarski-Knaster

The propositional Mu-Calculus is based on fixpoint operators. We start by restating the Tarski-Knaster theorem in terms of sets.

Given $f : Z \to Z$, we define $f^\alpha : Z \to Z$, the α-fold composition (iteration) of f as follows, where α ranges over On, the class of ordinals.

- $f^0(A) = A$
- $f^{\alpha+1}(A) = f(f^\alpha(A))$
- $f^\alpha(A) = \{x : \langle \exists \beta :: \langle \forall \gamma : \beta \prec \gamma \prec \alpha : x \in f^\gamma(A) \rangle \rangle\}$, when α is a limit ordinal.

If $f.x = x$ we say that x is a *fixpoint* of f. If f is a monotonic function on the powerset of a set, then by the following version of the Tarski-Knaster theorem, it has a least and greatest fixpoint, denoted by $\mu.f$ and $\nu.f$, respectively.

Tarski-Knaster

Theorem 2 *Let* $f : \mathcal{P}(S) \to \mathcal{P}(S)$ *such that* $a \subseteq b \quad \Rightarrow \quad f.a \subseteq f.b$ *for all* $a, b \in S$. *Then*

1. $\mu.f \;=\; \langle \cap b : b \subseteq S \;\wedge\; f.b \subseteq b : b \rangle \;=\; \langle \cup \alpha \in On :: f^{\alpha}(\emptyset) \rangle$, *and*
2. $\nu.f \;=\; \langle \cup b : b \subseteq S \;\wedge\; b \subseteq f.b : b \rangle \;=\; \langle \cap \alpha \in On :: f^{\alpha}(S) \rangle$,

We say that x is a *pre-fixpoint* of f iff $x \subseteq f.x$; x is a *post-fixpoint* iff $f.x \subseteq x$. The Tarski-Knaster theorem tells us that $\mu.f$ is below all post-fixpoints and that $\nu.f$ is above all pre-fixpoints.

We can replace On by the set of ordinals of cardinality at most $|S|$. When the state space is finite, this gives us an algorithm for computing least and greatest fixpoints. Notice that by the monotonicity of f, $\alpha \preccurlyeq \beta \quad \Rightarrow \quad f^{\alpha}(\emptyset) \subseteq f^{\beta}(\emptyset) \quad \wedge \quad f^{\beta}(S) \subseteq f^{\alpha}(S)$. Therefore, we can compute $\mu.f$ by applying f to $\emptyset$ until we reach a fixpoint; similarly, we can compute $\nu.f$ by applying f to S until we reach a fixpoint.

We now describe the Mu-Calculus. We are really describing a class of calculi parameterized by a set of variables, VAR, and a language for denoting predicates on labels. The predicate denoted by expression e is written $(\!|e|\!)$. The formulas of the Mu-Calculus are defined recursively as follows.

Mu-Calculus

- The syntax of the mu-calculus:
  - *e,* where *e* is an *expression*
  - $f \wedge g$ and $\neg f$, where *f*, *g* are formulas
  - **EX** *f*, where *f* is a formula
  - Y, where Y is a *variable*
  - μY *f* and νY *f* where Y is a variable & *f* is a formula, monotone in Y
- Fixpoint semantic characterization (V,W: Vars $\rightarrow \wp$(S))
  - $\langle\!\langle e \rangle\!\rangle_{M,V} = \{ s \in S: L.s \in [\![e]\!] \}$
  - $\langle\!\langle \neg f \rangle\!\rangle_{M,V} = S \setminus \langle\!\langle f \rangle\!\rangle_{M,V}$
  - $\langle\!\langle f \wedge g \rangle\!\rangle_{M,V} = \langle\!\langle f \rangle\!\rangle_{M,V} \cap \langle\!\langle g \rangle\!\rangle_{M,V}$
  - $\langle\!\langle \mathbf{EX}\ f \rangle\!\rangle_{M,V} = \{ s \in S: \langle \exists u \in S :: s \dashrightarrow u \wedge u \in \langle\!\langle f \rangle\!\rangle_{M,V} \rangle \}$
  - $\langle\!\langle Y \rangle\!\rangle_{M,V} = V.Y$
  - $\langle\!\langle \mu Y\ f \rangle\!\rangle_{M,V} = \mu.g$ where $g.y = \langle\!\langle f \rangle\!\rangle_{M,W}$ & W=V, except W.Y=*y*
  - $\langle\!\langle \nu Y\ f \rangle\!\rangle_{M,V} = \nu.g$ where $g.y = \langle\!\langle f \rangle\!\rangle_{M,W}$ & W=V, except W.Y=*y*

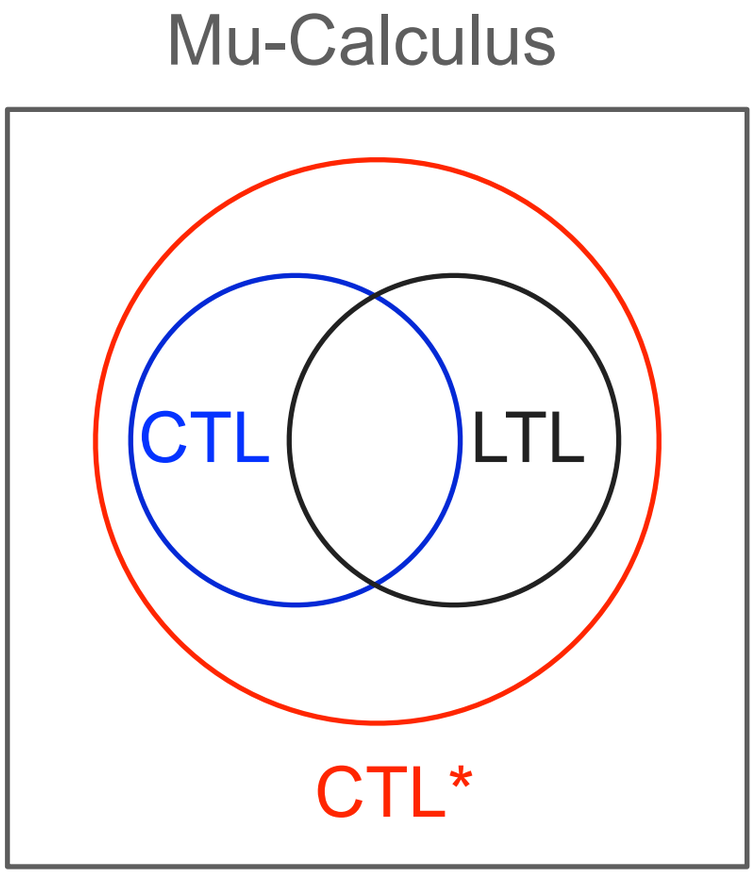
Mu-Calculus Examples

- $\langle\!\langle \mu Y\ f \rangle\!\rangle_{M,V} = \mu.g$ where $g.y = \langle\!\langle f \rangle\!\rangle_{M,W}$ & W=V, except W.Y=y
- $\langle\!\langle \nu Y\ f \rangle\!\rangle_{M,V} = \nu.g$ where $g.y = \langle\!\langle f \rangle\!\rangle_{M,W}$ & W=V, except W.Y=y
  - **EF** p = $\mu Y(p \vee \mathbf{EX}\ Y)$
  - **EG** p = $\nu Y(p \wedge \mathbf{EX}\ Y)$
  - **AF** p = $\mu Y(p \vee \mathbf{AX}\ Y)$
  - **AG** p = $\nu Y(p \wedge \mathbf{AX}\ Y)$
  - ✔ **E**(f **U** g) = $\mu Y(g \vee (f \wedge \mathbf{EX}\ Y))$
  - ✔ **A**(f **U** g) = $\mu Y(g \vee (f \wedge \mathbf{AX}\ Y))$
  - **EGF** p = $\nu Y\ \mu Z[\mathbf{EX}\ (Z \vee (p \wedge Y))]$

Can translate CTL to Mu-Calculus

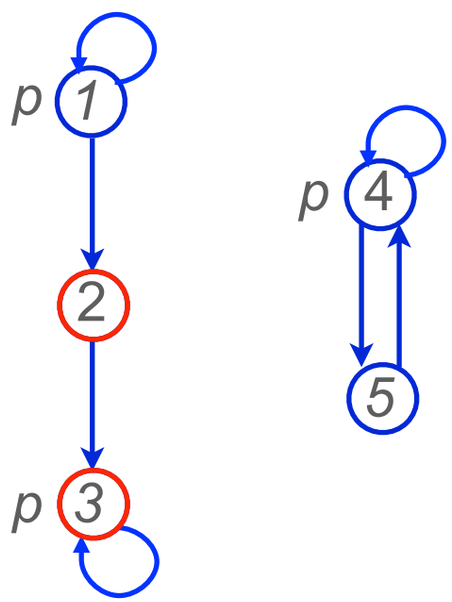
Symbolic Model Checking

- The mu-calculus subsumes CTL*
- Fixpoints can be computed with BDDs
- Symbolic Model Checking:
  - Convert Initial states to BDD
  - Convert transition relation to BDD
  - Convert CTL to mu-calculus
  - Use BDDs to check formula



Mu-Calculus Model Checking

Transition System M



$\langle\!\langle p \rangle\!\rangle_M = \{1, 3, 4\}$

$\langle\!\langle \mathbf{AG}\ p \rangle\!\rangle_{M\{1,2,3,4,5\}} = p \wedge \mathbf{AX}\ \{1,2,3,4,5\} = \{1, 3, 4\}$

$\langle\!\langle \mathbf{AG}\ p \rangle\!\rangle_{M\{1,3,4\}} = p \wedge \mathbf{AX}\ \{1,3,4\} = \{3\}$

$\langle\!\langle \mathbf{AG}\ p \rangle\!\rangle_{M\{3\}} = p \wedge \mathbf{AX}\ \{3\} = \{3\}$

$\langle\!\langle \mathbf{AF}(\mathbf{AG}\ p) \rangle\!\rangle_{M\{\}} = (\mathbf{AG}\ p) \vee \mathbf{AX}\ \{\} = \{3\}$

$\langle\!\langle \mathbf{AF}(\mathbf{AG}\ p) \rangle\!\rangle_{M\{3\}} = \{3\} \vee \mathbf{AX}\ \{3\} = \{2,3\}$

$\langle\!\langle \mathbf{AF}(\mathbf{AG}\ p) \rangle\!\rangle_{M\{2,3\}} = \{3\} \vee \mathbf{AX}\ \{2,3\} = \{2,3\}$

Property: $f = \mathbf{AF}(\mathbf{AG}\ p)$

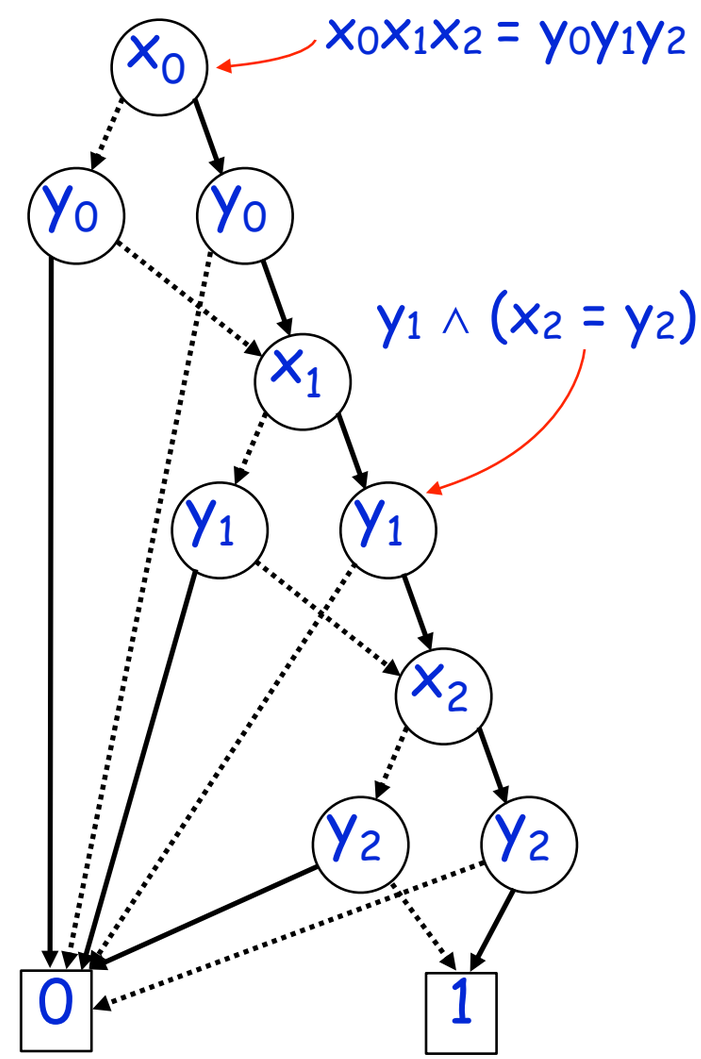
Which states in M satisfy f ?

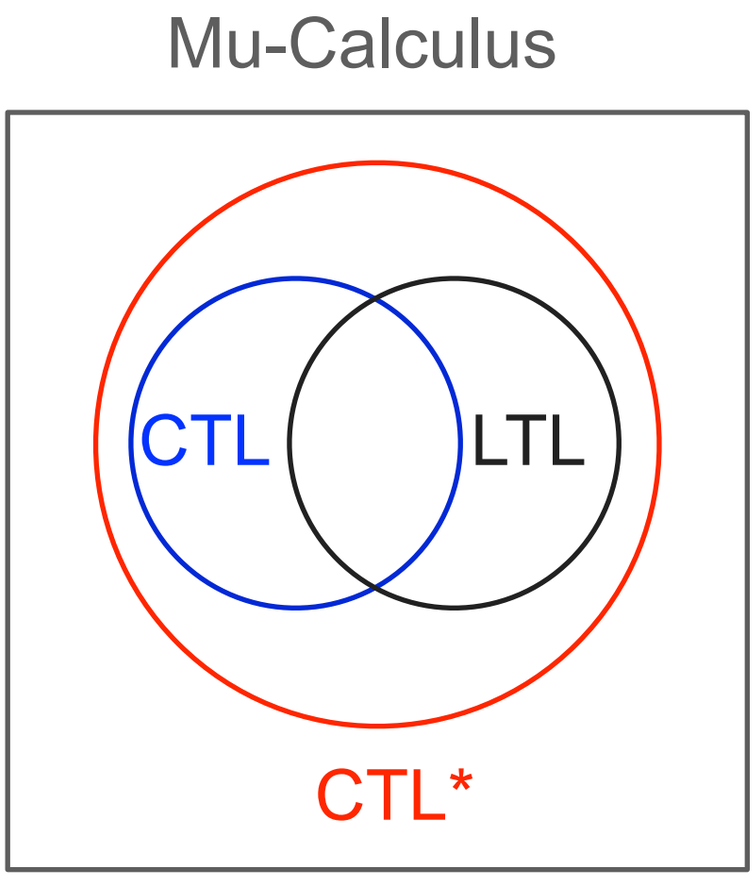
$\mathbf{AG}\ p = \nu Y(p \wedge \mathbf{AX}\ Y)$

$\mathbf{AF}\ p = \mu Y(p \vee \mathbf{AX}\ Y)$

Reachability w/ BDDs

- BDDs can represent sets
  - the language the BDD accepts
  - the set of assignments evaluating to 1
- BDDs can represent relations
  - use the characteristic function
  - can represent transition relation!
  - *eg*, T(*x*,*y*) holds iff *y* is a successor of *x*
- BDDs can be used to implement ∃
  - $\exists x{:}f = f_{x=0} \vee f_{x=1}$ (Shannon decomposition)
- BDDs can be used for reachability
  - $S_0(X)$ = Initial {X is a vector}
  - $S_{i+1}(X) = S_i(X) \vee \exists Y{:}[S_i(Y) \wedge T(Y, X)]$
  - $S_\infty = \cup_i S_i$ {stop when $S_i = S_{i+1}$: O(1)}



Model Checking Complexity

- Model checking complexity:
  - CTL: PTIME-complete, $O(|M| \times |f|)$
  - LTL, CTL*: PSPACE-complete
  - Mu-Calculus: $O(|M|^{d+1})$: d is alternation depth of formula
- Satisfiability complexity:
  - CTL: EXPTIME-complete
  - LTL: PSPACE-complete
  - CTL*: 2-EXPTIME-complete
  - Mu-Calculus: EXPTIME-complete